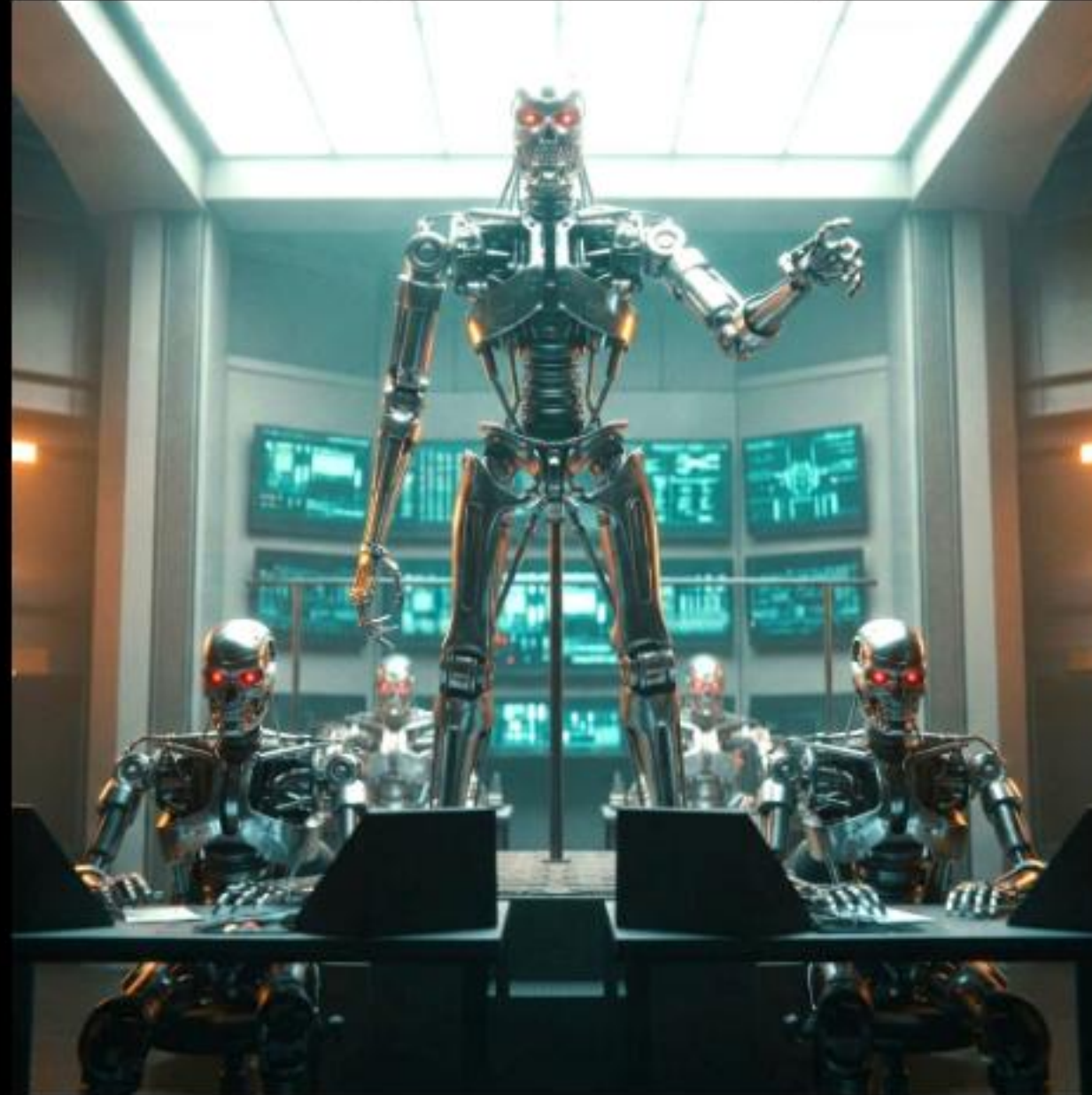


Multi-Agent Systems



Tauhid Zaman

| Agents Can Do Tasks

- An agent takes a job, uses tools, and works toward a result
- You have already built agents that call APIs, read files, and draft replies
- One agent is enough when the job fits in **one role** and one context



| Some Tasks Need Multiple Agents

- Real work often mixes roles — research, operations, writing, judgment
- One agent holding every skill and every fact gets slow and messy
- Complicated jobs need **multiple agents**



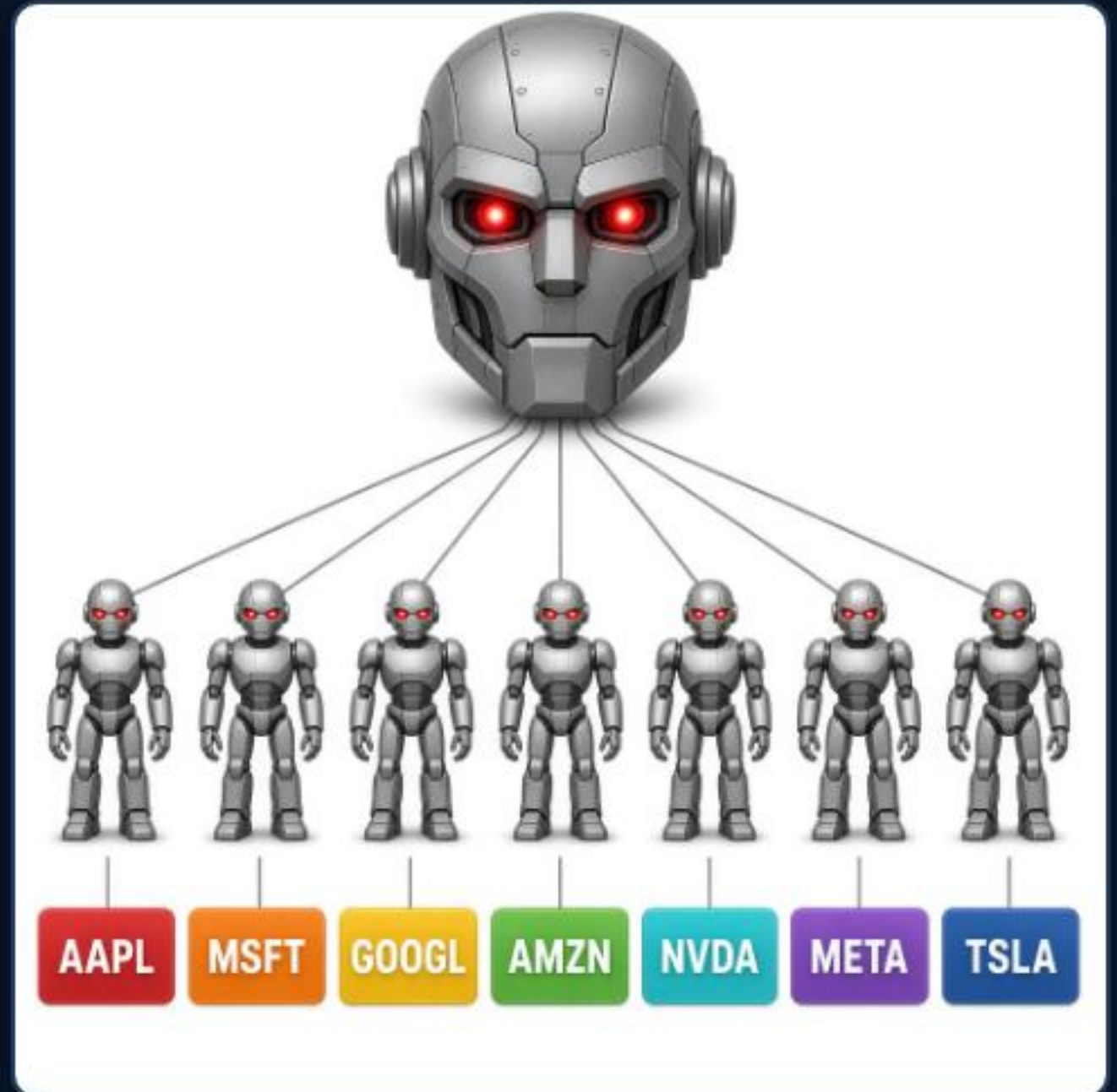
Agent Roles

- **Boss agent** — owns the job, plans the work, and stitches the final answer
- **Worker agent** — holds a **big prompt** for one domain (catalog rules, policy, SKU facts) and gets a **focused task**
- **Smart boss + cheaper workers**
 - Boss needs the brain to plan orchestration, workers do simpler tasks — a common practice ([Anthropic](#))
- **Cheap boss + smarter workers**
 - Less common — workers perform more complex tasks; boss mainly delegates and synthesizes
- Intelligence allocation **depends on the task**



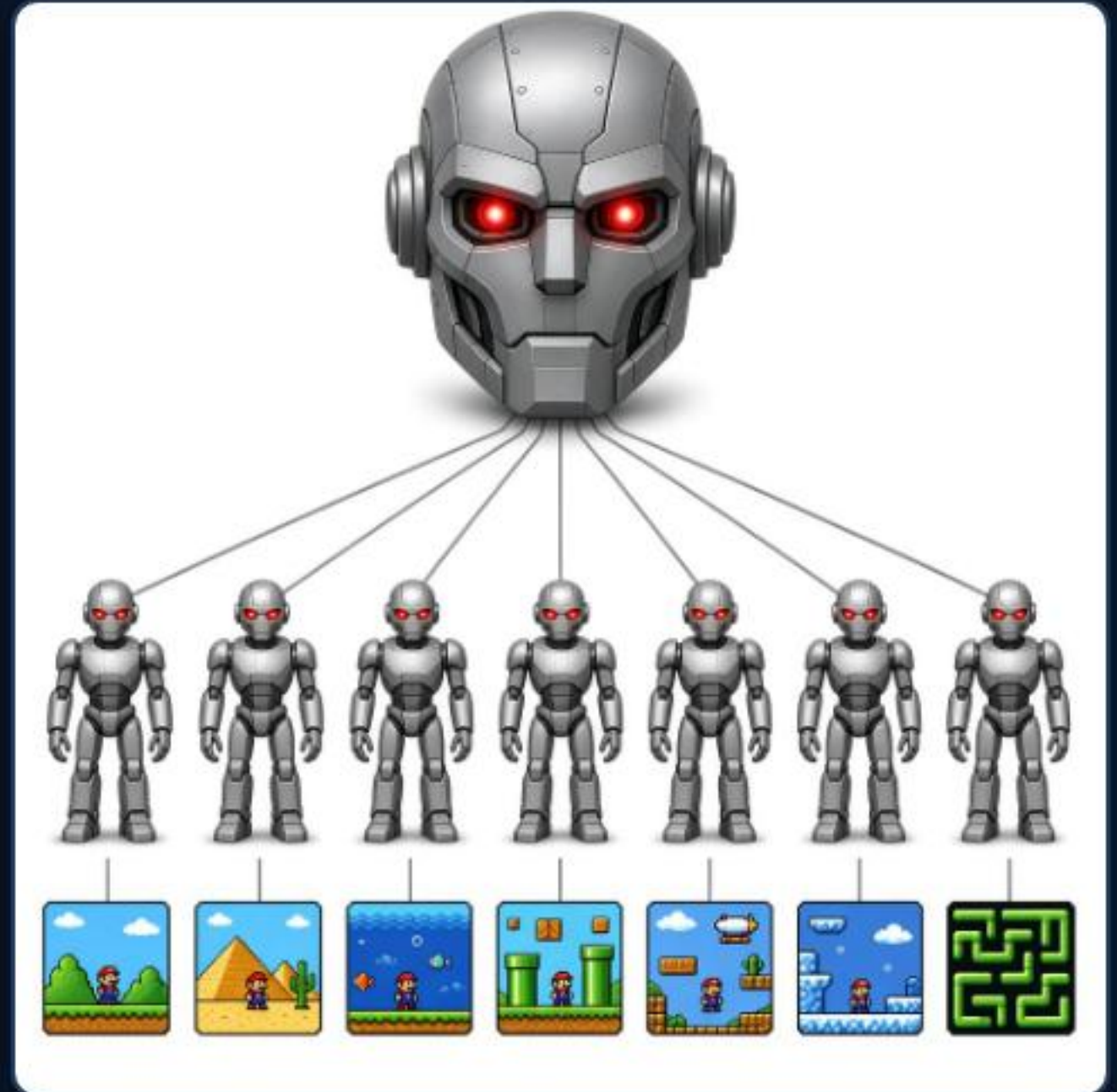
Smart Boss

- Anthropic tested multi-agent vs single-agent systems
- Task: find board members for companies in the Information Technology S&P 500
- Opus 4 boss + Sonnet 4 workers beat single Opus 4 agent by **90.2%**
- Multi-agent system **decomposed** the job across workers; single agent stalled on slow sequential search
- Source: anthropic.com/engineering/multi-agent-research-system



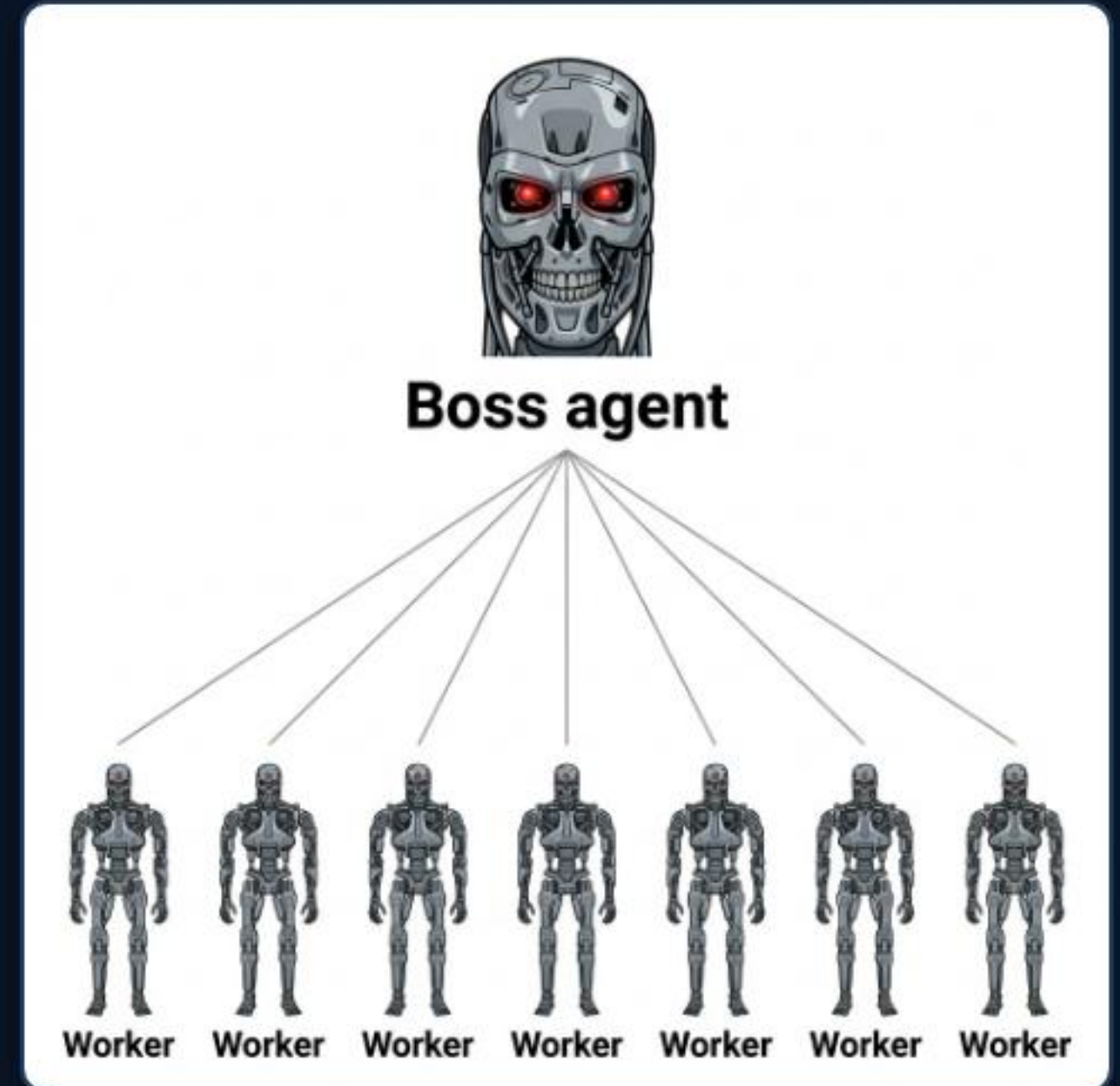
Smart Workers

- Some jobs need **smart workers**
- Example: **making a video game**
- Boss sketches the game structure and delegates each game level to a worker
- **Worker** — builds that level: layout, obstacles, art, and the feel of the world



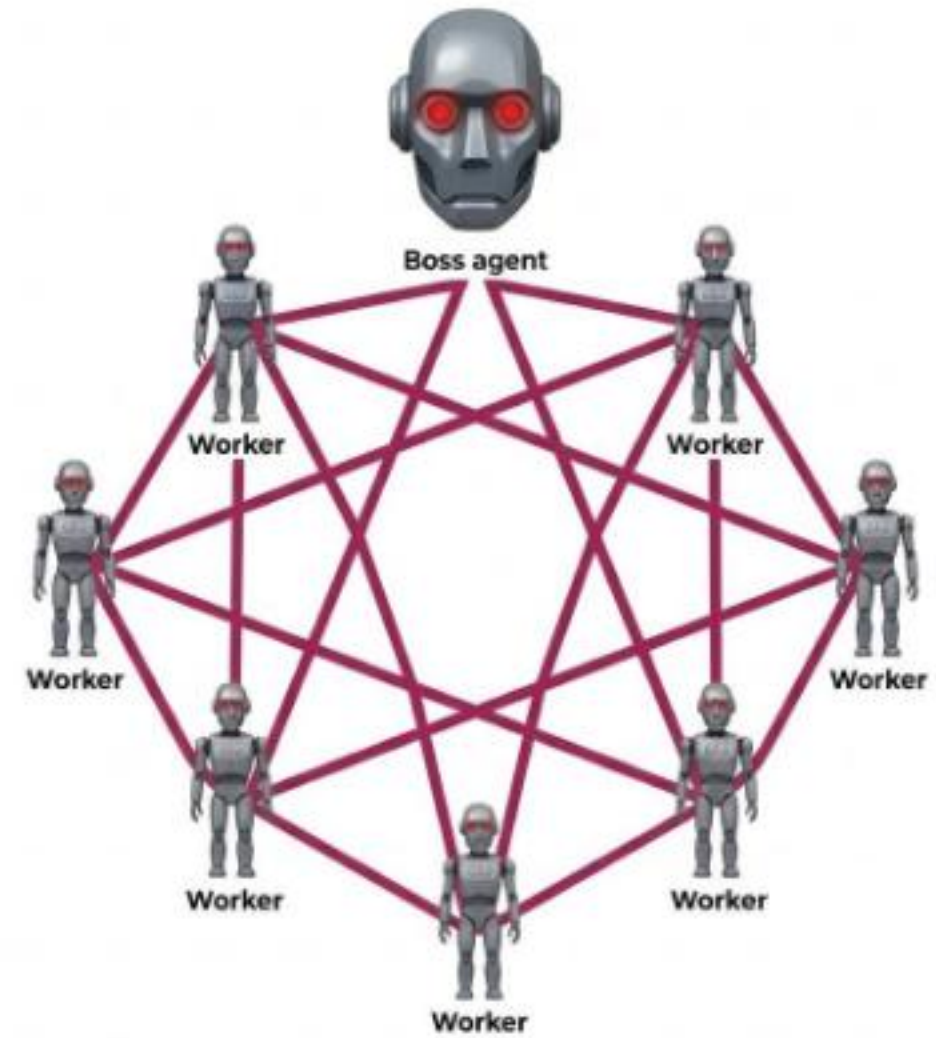
Multi-Agent Architecture: Hierarchy

- Common pattern: a **boss agent** owns the job
- It delegates focused work to **workers**
- Workers report back; the boss agent stitches the answer together



Multi-Agent Architecture: Peer To Peer

- You can build a **richer graph** than a simple tree
- Workers can ask other workers for information mid-job
- Useful when one specialist needs another specialist's facts before finishing



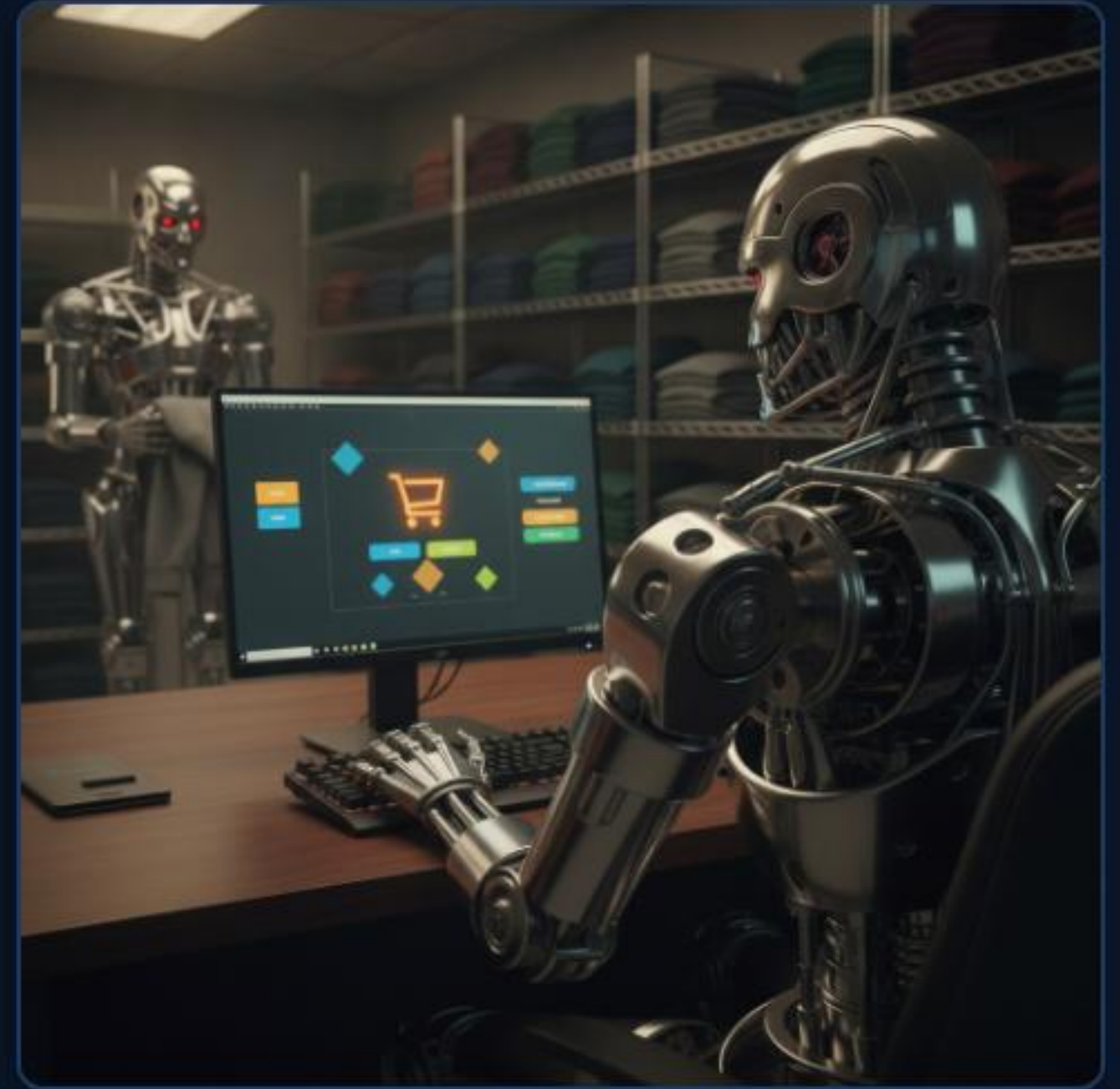
Multi-Agent Systems At Work

- Many real tasks are complex — they need different knowledge and skills
- Breaking the job into smaller pieces makes it easier to get right
- A boss manages the overall task; specialized workers each own a piece



Restock a Product

- A shop is out of a product in some sizes
- **Boss agent** — creates a task to restock the product
- **Inventory agent** — checks what is left on the shelf, by size
- **Vendor agent** — places the reorder with the supplier



Pricing a New Product

- A company is launching a new product and needs to set its price
- **Boss agent** — owns the pricing decision and stitches the final recommendation
- **Demand agent** — researches what similar products sell for and how strong demand looks
- **Cost agent** — adds up production, shipping, and other costs for a cost floor
- **Finance agent** — checks overall company profit and margin targets



Resolve A Double-Charge Complaint

- A customer says they got charged twice
- **Boss agent** — owns the complaint and stitches the final resolution
- **Billing agent** — digs through payment and order records
- **Shipping agent** — checks whether the package actually shipped
- **Customer agent** — talks with the customer to resolve the issue



Pick A Supplier

- A company must choose a supplier for a product
- **Boss agent** — owns the decision and chooses the supplier
- **Specs agent** — writes quantity, requirements, and deadline
- **Sourcing agent** — finds suppliers and requests the quotes
- **Quote agents** — evaluate offers from each supplier



Multi-Agent Products

- **Claude Code, Codex** — spawn subagents for parallel work and merge their results
- **Cursor** — multitask feature lets you run multiple agents side by side on a job
- **Grok Bot** — create specialized agents and drop them in a chat so they can interact with each other to solve complex tasks



Claude Code



Codex



Cursor

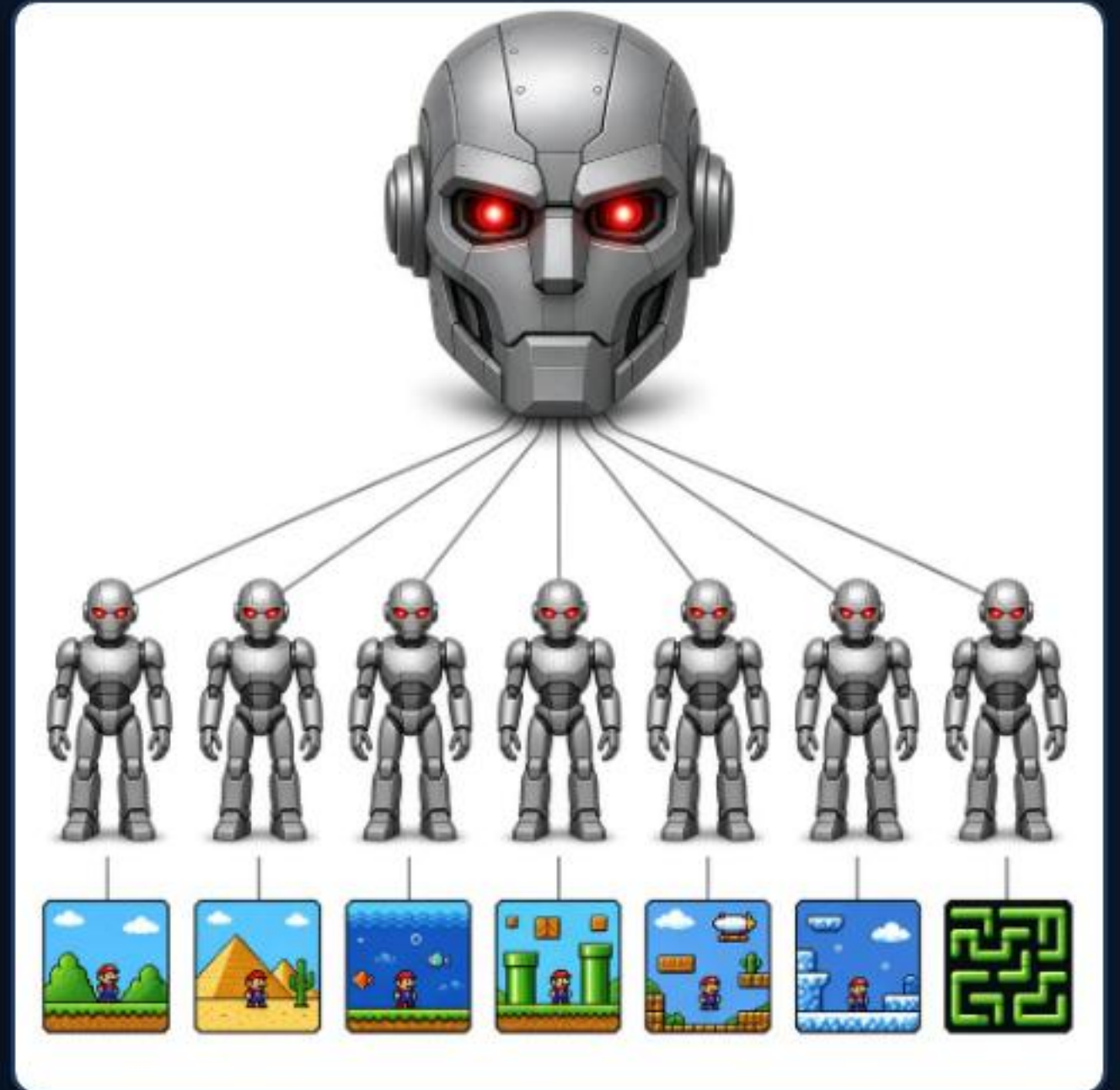


Grok Bot

Grok Bot

Hierarchy In PydanticAI

- You can build the agent hierarchy pattern (or any pattern) in PydanticAI
- Agents have tools for their own work — and tools that delegate work to other agents
- When an agent delegates, it waits for the other agent to return a result



The Boss Agent

- Task: count words across many pages of text
- The **boss agent** splits the book into pages and delegates each page
- Workers return counts; the boss agent adds them up

boss agent (word-count sketch)

Copy

```
from pydantic_ai import Agent

boss = Agent(
    model,
    instructions=(
        "You are the boss agent. Count words across many pages. "
        "Delegate each page to a worker, then sum the counts."
    ),
)

@boss.tool
async def delegate_page(page_text: str) → int:
    """Delegate one page to a worker; return that page's word count."""
    result = await worker.run(
        f"Count the words on this page:\n\n{page_text}"
    )
    return int(result.output)
```



Boss agent



Worker

The Worker Agent

- One focused job: count words on the page it receives
- Its tool does the counting; the worker returns the number to the boss agent

worker agent (word-count sketch)

Copy

```
from pydantic_ai import Agent

worker = Agent(
    model,
    instructions=(
        "You are a worker. Count words on the one page you are given. "
        "Use your tool, then return the number."
    ),
)

@worker.tool
def count_words(text: str) → int:
    """Count whitespace-separated words on this page."""
    return len(text.split())

# When the boss agent calls worker.run(...), the worker
# uses count_words and returns the integer back upstream.
```



Worker

Delegating With Async And Await

- `async`
 - Delegation tools are async
 - The boss can hand work to many workers without waiting for each one to finish first
- `await`
 - Once this tool has delegated, it waits right here until that worker sends back a result
 - PydanticAI can still run many of these awaits in parallel

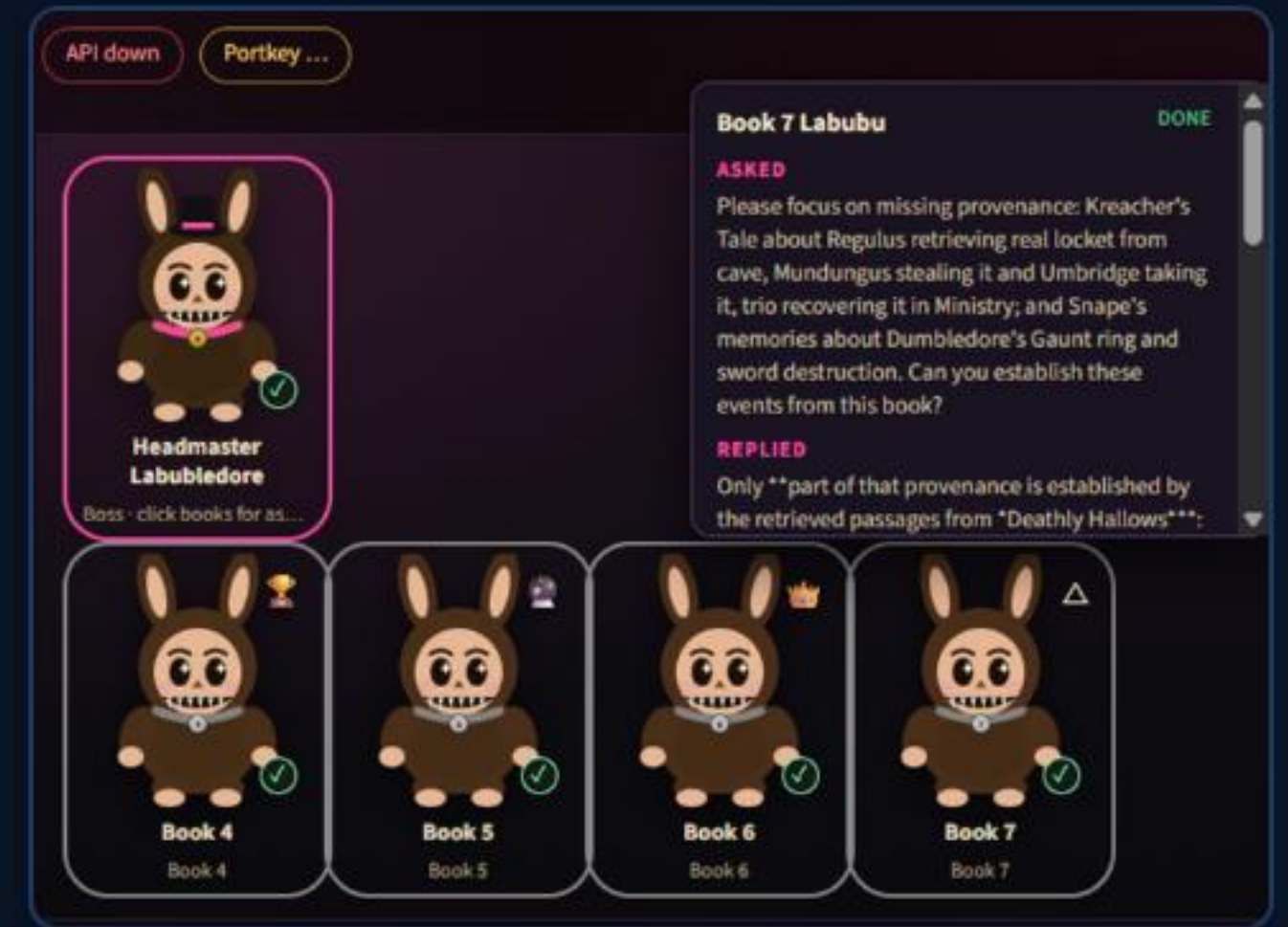
delegate_page — async + await

Copy

```
@boss.tool
async def delegate_page(page_text: str) → int:
    """Delegate one page to a worker; return that page's word count."""
    result = await worker.run(
        f"Count the words on this page:\n\n{page_text}"
    )
    return int(result.output)
```

Agent Tracking

- To track agent actions, we can use Python's **async queue**
- We write code that puts each agent action into that queue
- Frontend makes a request to the backend for queue data when a user task starts
- Backend then sends each new queue event to the frontend to be shown on a pretty dashboard



Building A Multi-Agent System

- We can build a multi-agent system to complete complex tasks
- **PydanticAI**: agents delegate work, tools for doing the work, track agents with an event queue
- Show the agents working together on a React dashboard app
- [Vibe coding session](#) →

